Capitolo 8

Design pattern strutturali

8.1 Comporre strutture e localizzare le dipendenze

I *design pattern strutturali* descrivono come comporre classi e oggetti per ottenere strutture più flessibili, riusabili ed estendibili. Mentre i pattern creazionali controllano come ottenere gli oggetti, quelli strutturali controllano come collegarli: possono rendere compatibili due interfacce, nascondere un sottosistema, rappresentare una gerarchia ricorsiva, aggiungere responsabilità oppure separare due dimensioni indipendenti della progettazione.

In Adapter, Facade, Composite, Decorator e Bridge il client, cioè il codice che richiede un servizio, deve dipendere dal contratto necessario e non dai dettagli che possono cambiare. Il ricorso a questi pattern non è automatico: ogni astrazione aggiunge classi, riferimenti e livelli di delega; è giustificato quando esiste il problema strutturale che essa risolve.

I cinque pattern trattati agiscono su problemi distinti:

Adapter converte l'interfaccia disponibile nell'interfaccia attesa dal client; risolve quindi un'incompatibilità fra contratti.

Facade offre un'unica interfaccia di alto livello a un sottosistema formato da più classi; riduce la complessità che il client deve conoscere.

Composite rappresenta una struttura ricorsiva parte–tutto e consente di trattare allo stesso modo un elemento semplice e un contenitore di elementi.

Decorator avvolge un oggetto per aggiungergli dati o comportamento dinamicamente, mantenendo il contratto dell'oggetto avvolto.

Bridge separa un'astrazione dalla sua implementazione quando entrambe variano indipendentemente; le collega mediante composizione anziché fondere le variazioni nell'ereditarietà.

Un'**interfaccia** è il contratto di operazioni visibile a chi usa un oggetto; una *composizione*, nel senso della progettazione a oggetti, è il possesso di un riferimento a un altro oggetto per delegargli parte del lavoro. La **delega** è proprio l'inoltro di una richiesta a tale oggetto. Queste definizioni non vanno confuse con la composizione UML, che esprime invece un vincolo di ciclo di vita fra intero e parte.

8.2 Adapter: tradurre un contratto incompatibile

L'intento di **Adapter** è trasformare l'interfaccia di una classe nell'interfaccia attesa dai client, permettendo così la collaborazione di classi altrimenti incompatibili. Il caso tipico è una classe di libreria utile che però non si può usare direttamente: i nomi dei metodi, il numero o tipo dei parametri, il tipo restituito oppure la sequenza di chiamate non coincidono con il contratto già usato dall'applicazione. Non si può, o non conviene, modificare né la libreria né tutti i chiamanti.

Nella variante *Object Adapter*, basata sulla composizione, i ruoli sono:

Target l'interfaccia che il client si aspetta;

Client il chiamante che usa soltanto oggetti conformi a Target;

Adaptee l'oggetto o la classe di libreria con l'interfaccia incompatibile;

Adapter l'oggetto che implementa Target, conserva un riferimento all'Adaptee e traduce la richiesta invocandone le operazioni appropriate.

Il client invoca dunque, per esempio, `target.request()`; Adapter converte argomenti e risultato quando necessario e invoca il metodo dell'Adaptee. Il client vede Adapter come un Target, non come una libreria. Target resta un contratto autonomo: se coincidesse con Adapter, il client sarebbe accoppiato all'adattamento e una modifica locale si propagherebbe ai chiamanti.

Nella variante *Class Adapter*, basata sull'ereditarietà, Adapter estende Adaptee e implementa Target: può chiamare i metodi ereditati senza mantenere un riferimento. L'Adaptee è però fissato nella gerarchia, quindi non è selezionabile a runtime. Questa forma può essere un *adapter a due vie*: poiché espone sia Target sia l'interfaccia ereditata di Adaptee, può essere passato a codice che si aspetta l'uno o l'altro contratto. Il termine "due vie" non significa che ogni Object Adapter abbia automaticamente questa proprietà.

Adapter mantiene indipendenti client e libreria: sostituire l'Adaptee richiede di intervenire nell'adattatore, non nel client. Può inoltre aggiungere controlli di *precondizione*, cioè condizioni da soddisfare prima dell'operazione, e di *postcondizione*, cioè proprietà garantite al termine se l'operazione riesce. Un Object Adapter può istanziare pigramente l'Adaptee alla prima invocazione che ne richieda l'uso (*lazy initialisation*).

Il costo è un livello di indirettezza: a ogni invocazione del client corrisponde almeno un'invocazione dell'adattatore, con possibile rallentamento e minore immediatezza del codice.

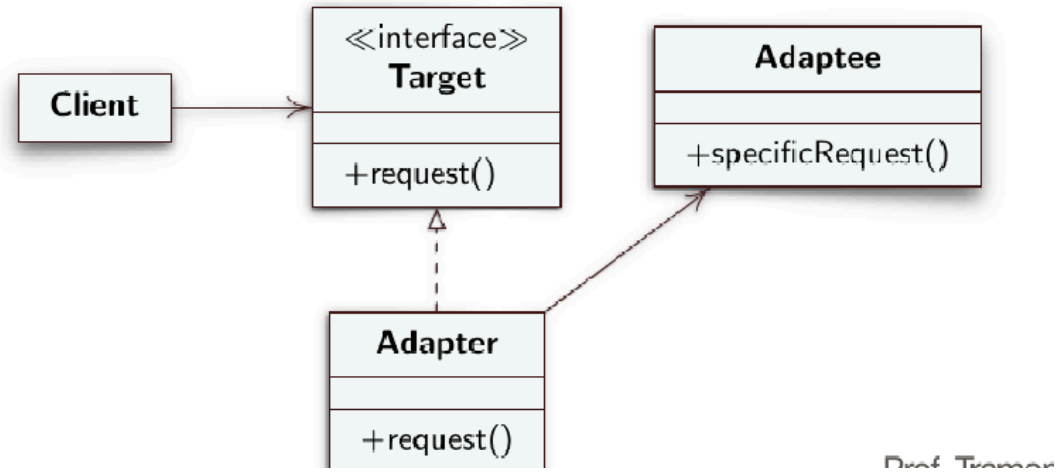


Figura 8.1: Object Adapter: Target è separato dalla libreria Adaptee.

Per supportare una nuova versione incompatibile della libreria si può mantenere l'adattatore precedente e aggiungerne uno per la versione nuova. Il client continua a usare Target, mentre il punto di configurazione seleziona l'adattatore necessario e può ripristinare quello precedente.

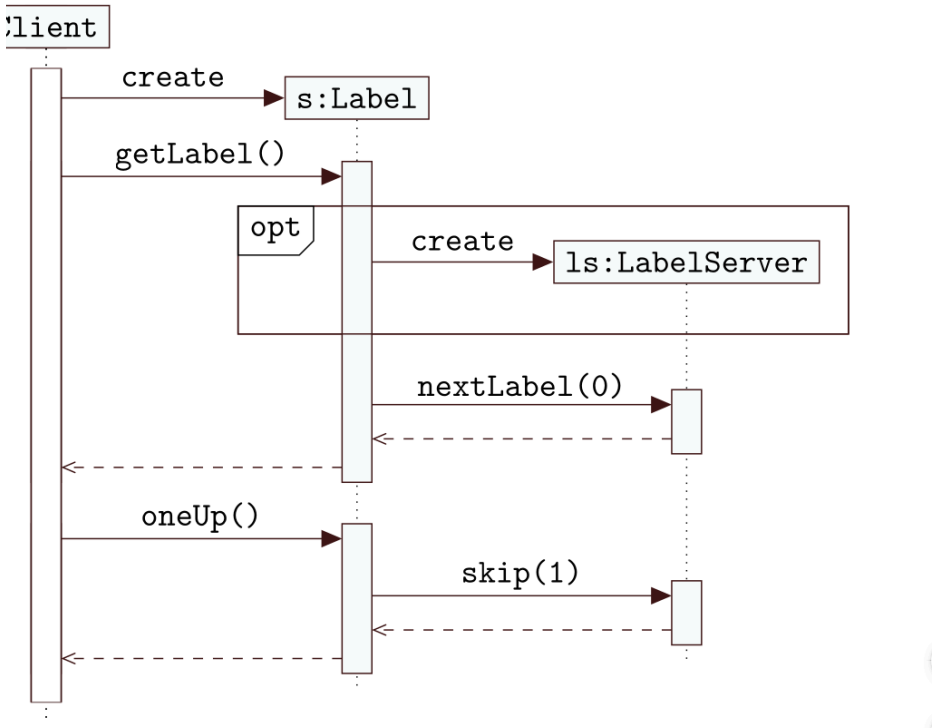


Figura 8.2: `Label` crea `LabelServer` solo quando necessario; il frame `opt` esprime il controllo opzionale.

Con la composizione, creare Adapter e creare Adaptee sono momenti distinti: il secondo può essere rimandato alla prima richiesta. Con un Class Adapter l'oggetto contiene invece anche la parte ereditata, inizializzata durante la sua stessa costruzione. Esporre entrambi i protocolli è possibile anche con composizione, se si progettano appositamente i due contratti. In Java

ciò non rende un Object Adapter sottotipo della classe Adaptee: aggiungere metodi con nomi uguali non equivale a ereditarne il tipo. La compatibilità va verificata rispetto ai tipi richiesti dai due client.

8.2.1 Delega nella sequenza delle chiamate

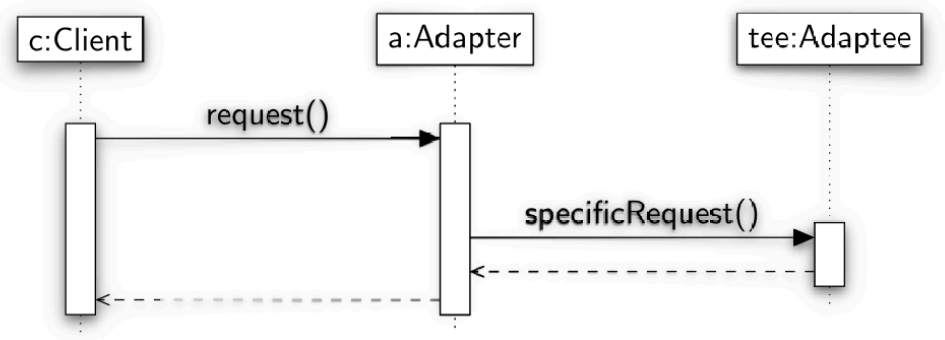


Figura 8.3: Il client invoca request sul Target; Adapter traduce la chiamata in specificRequest sull'Adaptee.

L'indirezione aggiunge una chiamata, ma mantiene separati il contratto dell'applicazione e quello della libreria. I controlli di precondizione avvengono prima della delega e quelli di postcondizione dopo il risultato; devono conservare le garanzie promesse dal Target.

8.2.2 Esempio Label e LabelC

`Label` svolge il ruolo di Object Adapter: adatta le chiamate dell'interfaccia che il client usa ai metodi di un oggetto di libreria. L'adattatore conserva l'Adaptee come campo e, nell'operazione Target, esegue la conversione necessaria prima di delegare. La versione `LabelC` è invece Class Adapter: è sottoclasse dell'Adaptee e implementa Target. L'esempio mostra il punto da verificare nella scelta: la prima versione può ricevere un oggetto adattato, la seconda lega Adapter alla sua superclasse.

```
interface ILabel {
    String getLabel(); String getPrefix(); void oneUp();
}
class LabelServer {
    private int labelNum = 1;
    private final String[] labelPrefix;
    LabelServer(String prefix) {
        labelPrefix = new String[]{prefix, "", ""};
    }
    public String nextLabel(int i) {
        if (checkRange(i)) return labelPrefix[i] + labelNum++;
```

```
        return null;
    }
    public String strip(int i) {
        return checkRange(i) ? labelPrefix[i] : null;
    }
    public void skip(int count) { labelNum += count; }
    public void setPrefix(String prefix, int i) {
        if (checkRange(i)) labelPrefix[i] = prefix;
    }
    private boolean checkRange(int i) {
        return i >= 0 && i < labelPrefix.length;
    }
}
class Label implements ILabel {
    private LabelServer ls;
    private void init() {
        if (ls == null) ls = new LabelServer("MyLabel");
    }
    public String getLabel() { init(); return ls.nextLabel(0); }
    public String getPrefix() { init(); return ls.strip(0); }
    public void oneUp() { init(); ls.skip(1); }
}
class LabelC extends LabelServer implements ILabel {
    LabelC() { super("MyLabel"); }
    public String getLabel() { return nextLabel(0); }
    public String getPrefix() { return strip(0); }
    public void oneUp() { skip(1); }
}
```

`Label` traduce nomi e parametri: `getLabel` diventa `nextLabel(0)`, `getPrefix` diventa `strip(0)`, `oneUp` diventa `skip(1)`. `init` crea il server soltanto al primo uso; nel diagramma di sequenza questa creazione appartiene al frammento opzionale `opt [ls==null]`. Con `ILabel x=new Label()`, la prima `x.getLabel()` restituisce `MyLabel1`; `x.oneUp()` salta il numero 2 e la successiva etichetta è `MyLabel3`. Il prefisso è `MyLabel`. Il server condivide il contatore fra i tre indici di prefisso e rifiuta indici fuori da $0, 1, 2$.

`LabelC` realizza lo stesso Target ereditando il server: è Class Adapter, senza riferimento delegato né inizializzazione lazy. Può essere usato sia come `ILabel` sia come `LabelServer`; questa è la variante a due vie. L'Object Adapter espone soltanto il contratto che decide di pubblicare e può scegliere o sostituire l'istanza adattata.

8.3 Facade: accesso di alto livello a un sottosistema

L'intento di **Facade** è fornire un'interfaccia unificata a un insieme di interfacce di un sottosistema, definendo un punto di accesso di alto livello che ne renda l'uso più semplice. Un *sottosistema* è qui un gruppo di classi che realizza funzioni correlate. Quando il client deve conoscere molte classi e coordinare molte chiamate, anche individuare l'interfaccia essenziale può diventare difficile.

Facade è una classe che espone poche operazioni orientate all'uso del client e, al loro interno, crea o usa gli oggetti del sottosistema e ne coordina le invocazioni. Il client interagisce con Facade; Facade non trasforma un'interfaccia incompatibile, come Adapter, ma semplifica un

insieme di interfacce già esistenti. La complessità non scompare: la facciata la concentra e la incapsula, mentre il sottosistema può restare invariato.

Facade riduce l'accoppiamento fra client e sottosistema. Finché il contratto esterno rimane stabile, la facciata può gestire una modifica interna senza farla ricadere sul client. La riduzione delle ricompilazioni dovute alle dipendenze interne dipende dal linguaggio e dalla build; in C++, per esempio, gli header interni possono essere inclusi dalla sola implementazione della facciata.

L'accesso diretto al sottosistema può restare disponibile ai client avanzati. Se deve essere vietato, occorrono regole di visibilità effettive, come classi annidate private o accessi controllati tramite package e moduli: né l'annidamento da solo né la separazione in cartelle costituiscono un vincolo sufficiente.

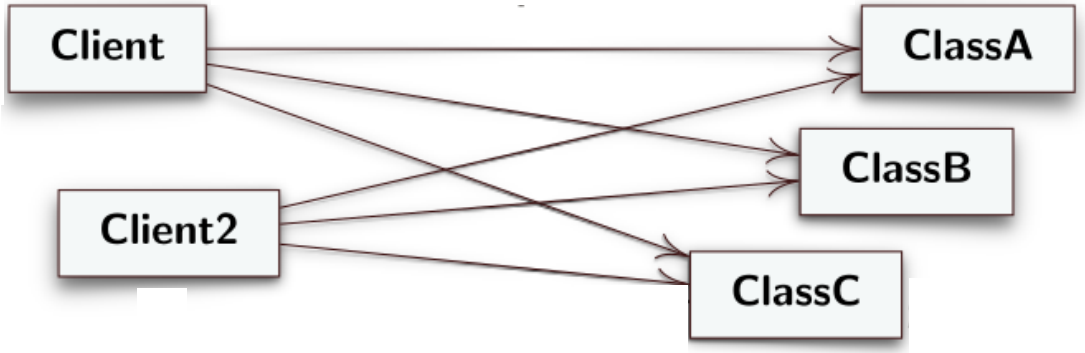


Figura 8.4: Prima della facciata ciascun client conosce più classi del sottosistema.

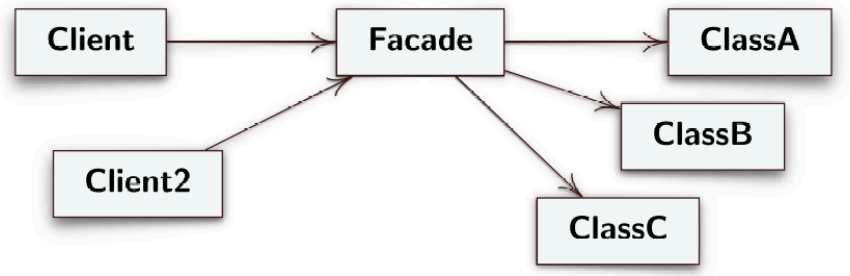


Figura 8.5: La facciata concentra l'accesso al sottosistema.

Se tre classi espongono rispettivamente tre, cinque e sei metodi, una facciata che si limiti a esporre tutte le quattordici operazioni non rende necessariamente più semplice usare il sottosistema. Deve offrire operazioni utili al livello del client e coordinare al proprio interno i dettagli.

8.3.1 Esempi Translator e Registro

Translator: coordinare due dizionari

La Facade espone `addEnglish(String)` e `multiPrinting()`; `getText()` appartiene invece alla classe interna `English`. I dizionari conservano quattro espressioni corrispondenti nella medesima posizione:

```
import java.util.List;

class English {
    private String text = " ";
    private final List<String> d =
        List.of("Alright", "Hello", "Understood", "Yes");
    boolean test(String s) { return d.contains(s); }
    void add(String s) { text = text + " " + s; }
    String getText() { return text; }
    int getIndex(String s) { return d.indexOf(s); }
    void printText() { System.out.println(text); }
}
class Italian {
    private String text = " ";
    private final List<String> d =
        List.of("Va bene", "Ciao", "Capito", "Sì");
    void add(int i) { text = text + " " + d.get(i); }
    void printText() { System.out.println(text); }
}
class Translator {
    private final English e = new English();
    private final Italian i = new Italian();
    void addEnglish(String s) {
        if (e.test(s)) {
            e.add(s);
            i.add(e.getIndex(s));
        }
    }
    void multiPrinting() {
        System.out.print("Italiano: "); i.printText();
        System.out.print("English: "); e.printText();
    }
}
```

Il client crea `Translator t`, chiama `t.addEnglish("Hello")` e `t.multiPrinting()`: ottiene Ciao nella riga italiana e Hello in quella inglese. La sequenza interna è controllo con `contains`, aggiunta inglese, ricerca dell'indice con `indexOf`, aggiunta italiana allo stesso indice. Una parola non ammessa non viene aggiunta a nessuno dei due testi.

La corrispondenza dipende dal fatto che le traduzioni occupino la stessa posizione nei due dizionari; non è una traduzione generale di frasi arbitrarie.

Registro: una richiesta di prestito, più responsabilità

Un prestito richiede un socio abilitato, un libro disponibile e il rispetto del limite dei prestiti contemporanei; cinque libri è un possibile limite applicativo, non una proprietà della Facade.

`Registro` riceve `ListaLibri` nel costruttore ed espone `prestaLib(socio,lib): boolean` e `trovaSocio(lib): String`.

Le responsabilità del sottosistema sono distinte: `Soci.verifica` controlla l'iscrizione; `BlockList.accettato` controlla l'abilitazione rispetto alla politica dei prestiti; `ListaLibri.libero` controlla la disponibilità e `rimuovi` aggiorna l'elenco; `Prestiti.registra` collega socio e libro e `trovaPersona` ricerca il titolare del prestito.

Il protocollo di `prestaLib` verifica dapprima `socio!=null` e `lib!=null`, restituendo falso in caso contrario. Quindi esegue, nell'ordine, `verifica`, `accettato`, `libero`, `rimuovi` e `registra`.

I primi controlli falliti interrompono il flusso; se la rimozione non riesce non si registra un prestito riuscito. Restituire subito il risultato nei casi negativi rende leggibile il percorso principale, senza annidare tutti gli `if`. `trovaSocio` delega alla ricerca nei prestiti.

Il client conosce l'operazione da richiedere, mentre Registro sa in quale ordine devono collaborare gli oggetti del sottosistema. Gestione di accessi concorrenti e atomicità fra rimozione e registrazione richiedono un contratto ulteriore e non sono garantite dalla sola presenza della Facade.

8.4 Composite: elementi semplici e composizioni ricorsive

L'intento di **Composite** è comporre oggetti in strutture ad albero per rappresentare gerarchie di parti e tutto, permettendo al client di trattare allo stesso modo gli oggetti singoli e le loro composizioni. La struttura ad albero è ricorsiva: un nodo contenitore può avere figli e ciascun figlio può essere a sua volta un contenitore. Senza Composite, il client di un file system dovrebbe distinguere con `if/else` file e cartelle e ripetere tale distinzione a ogni livello; analogo problema si presenta in un editor con figure semplici e gruppi di figure.

I partecipanti sono:

Component il contratto comune degli elementi semplici e composti; dichiara le operazioni che il client può richiedere a entrambi e può realizzare una sola volta quelle comuni alle sottoclassi;

Leaf un elemento semplice, sottoclasse o implementazione di Component, che esegue direttamente l'operazione;

Composite un elemento contenitore che mantiene una collezione di Component, implementa la gestione dei figli e delega ricorsivamente l'operazione a essi;

Client il codice che interagisce attraverso Component.

Per esempio, quando il client invoca `stampaContenuto()` su una Leaf, l'elemento stampa il proprio contenuto; se la riceve un Composite, questo la inoltra a ciascun figlio, eventualmente compiendo attività prima o dopo. La lista deve contenere elementi di tipo Component, non soltanto Leaf o un particolare Composite; altrimenti non potrebbe rappresentare una struttura di profondità arbitraria.

Composite e relazione parte–tutto

Precisazione

Il fatto che un oggetto contenga figli non impone, da solo, una relazione di composizione UML. Se le parti possono vivere indipendentemente, come un elemento trasferibile fra cartelle, la relazione può essere di aggregazione; se una parte appartiene esclusivamente al tutto e ne condivide il ciclo di vita, si usa la composizione. Per esempio, uno stato conservato come parte esclusiva di uno specifico ordine può essere modellato con rombo pieno. La scelta dipende dalla proprietà e dal ciclo di vita effettivi, non dal semplice uso di una lista.

Gestione dei figli: trasparenza e sicurezza

Le operazioni di gestione dei figli, quali `add()`, `remove()` e `getFigli()`, pongono un compromesso. Se sono dichiarate in Component, il client può usarle senza distinguere Leaf e Composite (**trasparenza**), ma su una Leaf non hanno significato e possono fallire o essere ignorate.

Se sono dichiarate solo in Composite, il compilatore impedisce di chiamarle su una Leaf (**sicurezza**), ma il client che deve usare queste operazioni perde la possibilità di trattare tutti gli elementi allo stesso modo. Un metodo quale `getComposite()`, che restituisce `null` per Leaf e l'oggetto stesso per Composite, non elimina il problema: al posto di `instanceof` usa un controllo su `null`.

Composite permette di aggiungere nuovi tipi di Leaf o Composite senza modificare i client che usano Component, purché le nuove classi ne rispettino il contratto. Questo realizza l'Open/Closed Principle: estendere i tipi disponibili può lasciare stabile il codice che li utilizza.

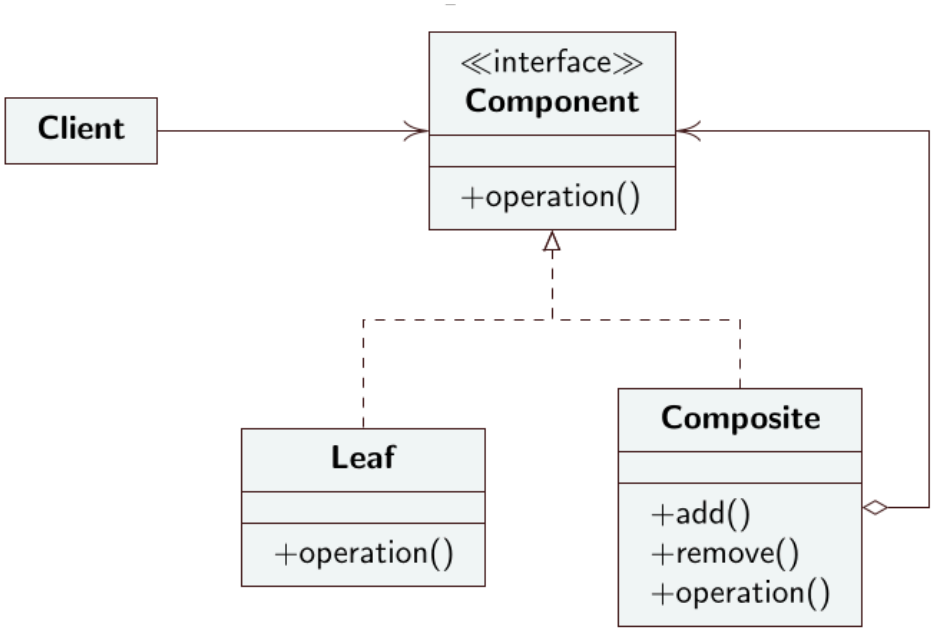


Figura 8.6: Component, Leaf e Composite e molteplicità dei figli.

La lista dei figli è nel Composite: inserirla in Component occuperebbe inutilmente spazio anche nelle foglie, che non la usano. Il riferimento al padre può appartenere invece al contratto

o a una base comune se tutti i nodi devono navigare verso l'alto. Quando si aggiunge, rimuove o sposta un nodo, relazione padre–figlio e cache degli antenati vanno mantenute coerenti. Una **cache** conserva un risultato già calcolato per riutilizzarlo.

Con una cache della dimensione, modificare una foglia richiede di invalidare il suo contenitore, poi il padre del contenitore, fino alla radice, per esempio mediante un'operazione `invalidaCache()` richiamata sul padre finché il riferimento non è nullo.

L'ordine dei figli può determinare l'ordine con cui si disegna, si stampa o si eseguono le operazioni; non va lasciato implicito quando modifica il risultato. Nel modello generico una collezione di Component ammette sia foglie sia altri compositi: una cartella che accetti soltanto file richiede un vincolo aggiuntivo. La limitazione è del modello scelto; non è un divieto generale di progettare varianti con contratti o tipi più restrittivi.

Una nuova Leaf `Shortcut` può essere usata dagli stessi client di File e Cartella se implementa le operazioni di Component. Il suo eventuale riferimento a un altro nodo richiede una politica di visita: seguire i collegamenti può introdurre cicli che non esistono nell'albero di contenimento.

Precisazione

Il rombo UML non dice che il metodo del contenitore debba soltanto inoltrare chiamate: l'operazione del Composite può aggiungere lavoro prima o dopo la visita. Aggregazione e composizione descrivono la relazione fra gli oggetti e il loro ciclo di vita, non il numero di istruzioni del metodo. Se una Leaf è realizzata tramite record, il nome di un componente non soddisfa automaticamente un metodo diverso dell'interfaccia: se il componente si chiama `nome` e Component richiede `descrizione()`, va implementato esplicitamente tale metodo.

8.4.1 Esempi: file, cartelle e figure

Un file system può distinguere `File` e `Cartella`, ma entrambi sono Component. Una cartella conserva Component e può quindi contenere file e ulteriori cartelle; invocare `stampaContenuto()` sulla radice percorre la struttura ricorsivamente. L'editor di figure è equivalente: una `Linea` o un `Box` è una Leaf, mentre un gruppo è Composite; `disegna()` o `sposta()` vale uniformemente per entrambi. Gli esempi chiariscono che "uniformemente" riguarda l'interfaccia del client, non implica che Leaf e Composite eseguano internamente lo stesso algoritmo.

Esercizio: completare un Composite con una Leaf record

Dato il contratto `descrizione()`, si deve realizzare una foglia `Auto` e un contenitore di elementi semplici o composti. Il record può implementare il metodo richiesto con un componente omonimo; se il componente ha un nome diverso, serve un'implementazione esplicita:

```
import java.util.*;

interface Elemento { String descrizione(); }
record Auto(String nome) implements Elemento {
    public String descrizione() { return nome; }
}
class Gruppo implements Elemento {
    private final List<Elemento> figli = new ArrayList<>();
```

```
    public void add(Elemento e) { figli.add(e); }
    public void remove(Elemento e) { figli.remove(e); }
    public String descrizione() {
        StringJoiner testo = new StringJoiner(", ", "[", "]");
        for (Elemento e : figli) testo.add(e.descrizione());
        return testo.toString();
    }
}
```

La scelta di parentesi e separatore rende osservabile la visita, senza cambiare i ruoli del pattern. Una foglia con nome `auto` restituisce quel testo; un gruppo che contiene tale foglia e un altro gruppo mostra il risultato annidato. La lista di `Elemento` permette profondità arbitraria. Le operazioni sui figli sono qui soltanto in `Gruppo`: il client dell'operazione `descrizione` è uniforme, quello che modifica la struttura deve conoscere il contenitore. Per mantenere un albero non si deve aggiungere un gruppo a sé stesso o a un proprio discendente.

8.5 Decorator: responsabilità aggiunte a un oggetto

L'intento di **Decorator** è aggiungere responsabilità a un singolo oggetto dinamicamente, offrendo un'alternativa flessibile alla creazione di sottoclassi. Una *responsabilità* è qui un dato o comportamento aggiuntivo. L'ereditarietà assegna una responsabilità alla classe a design-time: con più estensioni e combinazioni genera molte sottoclassi rigide. Decorator avvolge invece l'oggetto (*wrap*) in un altro oggetto conforme alla stessa interfaccia; le decorazioni possono essere aggiunte, sottratte o annidate a runtime.

I ruoli sono:

Component l'interfaccia comune a oggetti decorabili e decoratori;

ConcreteComponent l'oggetto di base, che implementa Component e non conosce chi lo avvolgerà;

Decorator una classe che implementa Component e conserva un riferimento a un Component; in genere inoltra a esso le richieste;

ConcreteDecorator una sottoclasse di Decorator che aggiunge stato o logica prima e/o dopo la delega.

La doppia relazione di Decorator con Component è essenziale: lo implementa, quindi il client può usare un decorator ovunque si aspetti Component; ne contiene uno, quindi può avvolgere sia un ConcreteComponent sia un altro ConcreteDecorator. Le richieste attraversano quindi gli strati di decorazione fino al componente di base; ciascuno strato può aggiungere il proprio comportamento prima o dopo la delega.

La composizione di decorazioni è ricorsiva e anche la stessa responsabilità può essere aggiunta più volte creando più istanze del medesimo ConcreteDecorator. Ciò evita classi di base gonfie e classi per ogni combinazione, favorendo il principio di Singola Responsabilità: ogni decoratore può avere un compito circoscritto.

I limiti sono importanti: l'oggetto risultante non ha né lo stesso tipo concreto né lo stesso riferimento dell'oggetto avvolto, dunque confronti basati su tipo o identità possono non funzionare come prima. Inoltre una lunga catena crea molti piccoli oggetti e deleghe; il debug e la comprensione richiedono di seguire l'ordine degli avvolgimenti.

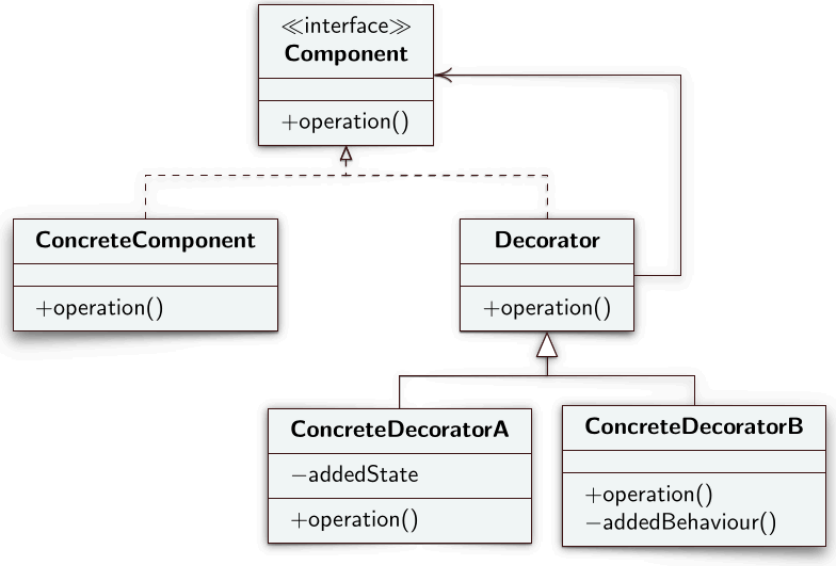


Figura 8.7: Il decoratore è un Component e contiene un Component.

L'aggiunta a runtime avviene componendo istanze di classi il cui codice esiste già: il pattern non genera nuovo codice dentro l'oggetto avvolto. Il ConcreteDecorator conserva il nome e il contratto dell'operazione di Component; può richiamare la delega della superclasse e aggiungere stato o comportamento. Decorator può essere astratto anche quando tutti i suoi metodi hanno un corpo: `abstract` impedisce l'istanziazione diretta e non obbliga a riscrivere i metodi concreti ereditati.

Ordine delle decorazioni

L'ordine di avvolgimento è significativo: applicare uno sconto prima di una maggiorazione può differire dall'applicarlo dopo.

Per rimuovere uno strato si deve rendere nuovamente raggiungibile il componente interno; eliminare uno strato interno può richiedere di ricostruire la catena esterna. Ripetere lo stesso decoratore è possibile, ma il contratto deve chiarire se l'effetto ripetuto abbia senso.

Riferimenti e identità degli oggetti

Un riferimento già conservato al componente base non viene aggiornato automaticamente quando un altro riferimento riceve il wrapper:

```
Component base = new Testo();
Component prima = base;
Component decorato = new Bordo(base);
// prima e' ancora il componente base.
// Le operazioni aggiuntive si ottengono usando decorato.
```

Per vedere la decorazione il client deve ricevere il riferimento al decoratore o un'indirezione stabile che sia stata aggiornata. La trasparenza dell'interfaccia non significa che si tratti dello stesso oggetto. Il decoratore dovrebbe dipendere dal contratto e dai risultati del componente, senza richiedere di conoscerne l'algoritmo interno; Se la cooperazione dipende troppo dai dettagli specifici degli oggetti, il contratto potrebbe essere insufficiente.

8.5.1 Esempio: bordo e catena di decorazioni

Per aggiungere un bordo a `Testo`, la soluzione rigida sarebbe introdurre una sottoclasse che possieda sempre il bordo. `DecoratorBordo` contiene invece un Component Testo, inoltra la richiesta e aggiunge il comportamento del bordo. Poiché Decorator implementa Component, la costruzione concettuale

```
Component c = new ConcreteDecoratorA(
                  new ConcreteDecoratorB(
                      new ConcreteComponent()));
```

è valida: il decoratore più esterno è ancora un Component. L'ordine è osservabile se le responsabilità eseguono attività prima o dopo la delega; una catena va quindi costruita deliberatamente e non confrontata con l'identità del componente interno.

Implementazione della delega e applicazione ai biglietti

```
interface Component { void operation(); }
class ConcreteComponent implements Component {
    public void operation() { System.out.print("componente"); }
}
abstract class Decorator implements Component {
    private final Component innerC;
    Decorator(Component c) { innerC = c; }
    public void operation() { innerC.operation(); }
}
class ConcreteDecorator extends Decorator {
    ConcreteDecorator(Component c) { super(c); }
    public void operation() {
        super.operation();
        addedBehavior();
    }
    private void addedBehavior() { System.out.print(" + aggiunta"); }
}
```

Il campo finale conserva il collaboratore; `super.operation()` raggiunge la delega già implementata nella base, che arriva al componente avvolto. La stampa scelta rende visibile l'ordine: prima il componente, poi l'aggiunta. Annidando due decoratori si esegue prima il componente e poi ciascuna aggiunta dall'interno verso l'esterno.

Per i biglietti il contratto `Prodotto` offre `printNome()` e `getCosto()`. `Biglietto` è il componente; `Decor` conserva un Prodotto e delega entrambe le operazioni.

`Formatta` modifica la presentazione di `printNome`, mentre `Sconto` modifica il risultato di `getCosto`. `Configura.getBiglietto` costruisce la catena base, `getBiglScontato` quella con

lo strato di sconto. Una configurazione `new Formatta(new Sconto(biglietto))` porta la richiesta di costo da Formatta a Sconto e poi a Biglietto, applicando lo sconto al ritorno. Il formato della stampa e la percentuale non sono fissati dalla struttura: devono essere parametri o regole del dominio. Questo esempio separa due responsabilità combinabili senza creare una sottoclasse per ciascuna combinazione.

8.6 Bridge: due gerarchie che variano indipendentemente

L'intento di **Bridge** è disaccoppiare un'astrazione dalla sua implementazione affinché le due possano variare indipendentemente. In questo pattern *astrazione* non significa necessariamente una classe astratta Java: è l'interfaccia di alto livello che dice *che cosa* il client vuole fare. *Implementazione* non significa soltanto una classe concreta: è l'insieme delle primitive di basso livello che dice *come* eseguirlo. Le due dimensioni sono *ortogonali* quando cambiano per ragioni indipendenti.

Se forme (rettangoli e cerchi) e piattaforme di disegno (P1 e P2) fossero fuse mediante sola ereditarietà, ogni combinazione richiederebbe una classe, quale `V1Rettang`. La scelta della piattaforma sarebbe permanente e non cambiabile a runtime. Bridge separa invece le due gerarchie: Forma conserva un riferimento a `DrawingAPI`, la cui interfaccia è implementata dalle piattaforme adattate. La composizione permette a un oggetto Forma di delegare le primitive alla piattaforma ricevuta e, se opportuno, sostituirla.

I partecipanti sono:

Abstraction definisce le operazioni di alto livello per il client e conserva un riferimento a Implementor; inoltra a esso il lavoro di basso livello;

RefinedAbstraction estende Abstraction con operazioni specifiche, continuando a usare l'Implementor ereditato;

Implementor definisce le operazioni primitive richieste dall'astrazione; la sua interfaccia non deve coincidere con quella di Abstraction;

ConcreteImplementor realizza le primitive per una particolare piattaforma.

Per il rettangolo, per esempio, RefinedAbstraction può chiamare più volte una primitiva di linea dell'Implementor per disegnare i quattro lati. La delega generica è `operation()` che richiama `impl.operationImpl()`: la prima appartiene all'astrazione, la seconda al contratto dell'implementazione. Il client deve conoscere Abstraction, non i ConcreteImplementor: ciò realizza *information hiding*, cioè nasconde i dettagli che non gli servono.

Bridge permette di aggiungere una forma senza modificare le piattaforme e una piattaforma senza modificare le forme, purché il contratto delle primitive sia sufficiente. Consente inoltre di configurare o sostituire l'implementazione a runtime. Quando piattaforme preesistenti offrono operazioni incompatibili, i ConcreteImplementor possono adattarle al contratto comune: Bridge organizza le dimensioni indipendenti, Adapter traduce le interfacce concrete.

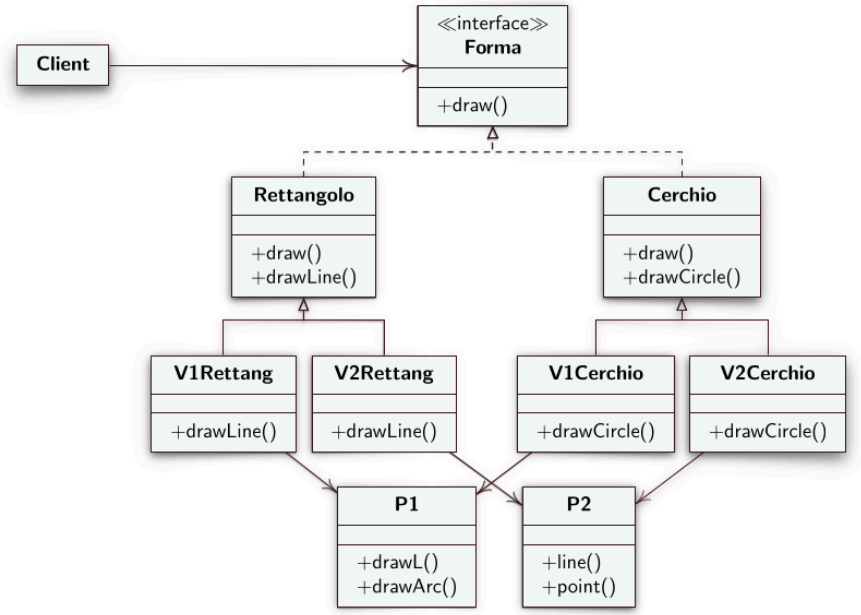


Figura 8.8: La struttura iniziale fonde forma e piattaforma in una classe per ogni combinazione.

Siano m il numero di varianti dell'astrazione, per esempio le forme, e n il numero di varianti dell'implementazione, per esempio le piattaforme. Fondere le due dimensioni in una gerarchia richiede una combinazione concreta per ogni coppia:

$$N_{\text{ereditarieta}} = mn. \tag{8.1}$$

Con Bridge le due gerarchie restano separate e il numero concettuale di classi varianti è:

$$N_{\text{bridge}} = m + n. \tag{8.2}$$

I valori $N_{\text{ereditarieta}}$ e N_{bridge} confrontano soltanto le varianti dovute alle due dimensioni, escludendo le interfacce e le classi base comuni. Con due forme e tre piattaforme, la prima soluzione richiede sei combinazioni; la seconda, cinque varianti distribuite sulle due gerarchie, oltre alle strutture comuni e agli eventuali adattatori. Il vantaggio principale resta l'indipendenza delle estensioni.

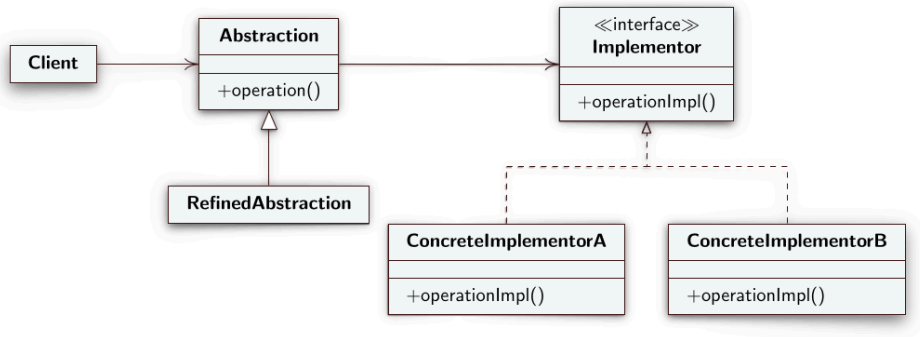


Figura 8.9: Abstraction mantiene un Implementor; le due gerarchie possono essere estese separatamente.

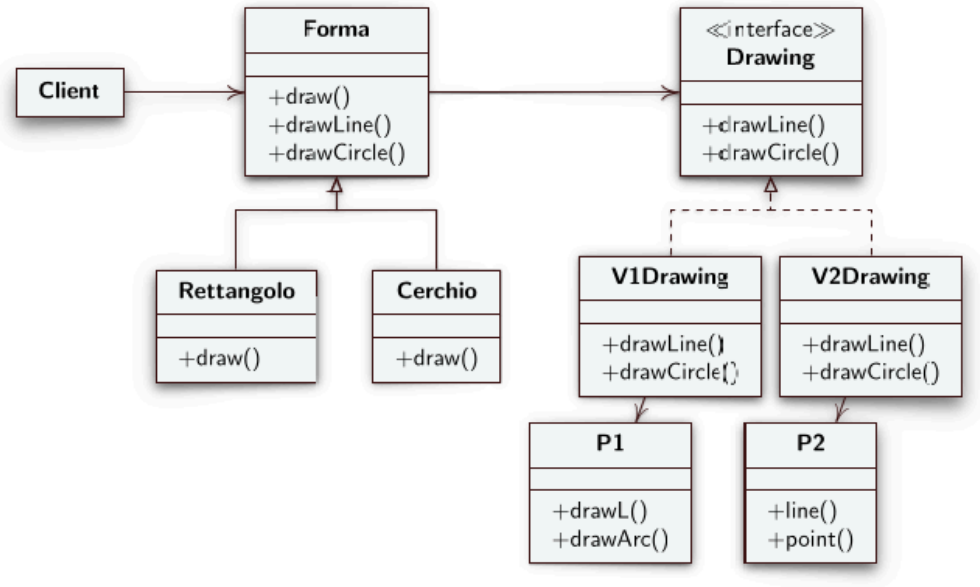


Figura 8.10: Le gerarchie Forma e Drawing sono collegate; V1Drawing e V2Drawing adattano le piattaforme P1 e P2.

`Drawing` espone `drawLine()` e `drawCircle()`; P1 dispone di `drawL()` e `drawArc()`, P2 di `line()` e `point()`. I ConcreteImplementor traducono queste primitive, svolgendo anche Adapter. Dal punto di vista dell'Adapter, Forma è Client, Drawing è Target, V1Drawing è Adapter e P1 è Adaptee. Il Bridge gestisce invece la variazione indipendente delle due gerarchie.

Le RefinedAbstraction possono avere operazioni specifiche; rimangono indipendenti dalle piattaforme finché le primitive di Implementor sono sufficienti. Se una nuova forma richiede

una primitiva non esprimibile con il contratto esistente, anche Implementor e le implementazioni devono evolvere: l'indipendenza non elimina la necessità di un contratto adeguato.

Lo stesso problema si presenta se Studente, Registro e Professore devono usare piattaforme AWT o WEB. Legare ogni astrazione alla piattaforma porta a sei varianti, che diventano nove con una terza piattaforma. Il Bridge rende espliciti i due motivi di variazione: entità del dominio e servizi della piattaforma.

8.6.1 Quattro lati espressi mediante una primitiva comune

```
interface Drawing {
    void drawLine(int x1, int y1, int x2, int y2);
}
class Forma {
    private Drawing impl;
    public void setImplementor(Drawing imp) { impl = imp; }
    protected void drawLine(int x1, int y1, int x2, int y2) {
        impl.drawLine(x1, y1, x2, y2);
    }
}
class Rettangolo extends Forma {
    private final int a, b, c, d;
    Rettangolo(int xi, int yi, int xf, int yf) {
        a = xi; b = yi; c = xf; d = yf;
    }
    public void draw() {
        drawLine(a,b,c,b); drawLine(a,b,a,d);
        drawLine(c,b,c,d); drawLine(a,d,c,d);
    }
}
```

Il client deve chiamare `setImplementor` con un Drawing valido prima di `draw`. `Rettangolo` conosce gli estremi geometrici e le quattro primitive; la concreta di Drawing traduce ciascuna primitiva nell'API grafica.

Cambiare backend mantenendo l'interfaccia non richiede di modificare il codice dell'astrazione né dei client che dipendono solo da essa. In Java ciò limita anche le dipendenze di compilazione: una modifica interna a una concreta non impone di riscrivere o ricompilare concettualmente tutte le forme; un cambiamento al contratto comune può invece propagarsi.

Una versione più estesa distingue anche i punti e i vertici: `Punto` espone `getX`, `getY` e `move`; `Rettangolo` e `Triangolo` conservano vertici accessibili con `getVertex` e realizzano lo spostamento agendo su di essi. `Forma` coordina `draw`, `drawLinea`, `drawPunto` e `setLook`; Drawing espone ora sia `drawLine` sia `drawPoint`. `V1Drawing` traduce verso `StdDraw`, `V2Drawing` verso la piattaforma alternativa. L'estensione geometrica e quella del backend restano due assi distinti; aggiungere una primitiva comune richiede invece di adeguare il contratto e i suoi implementatori.

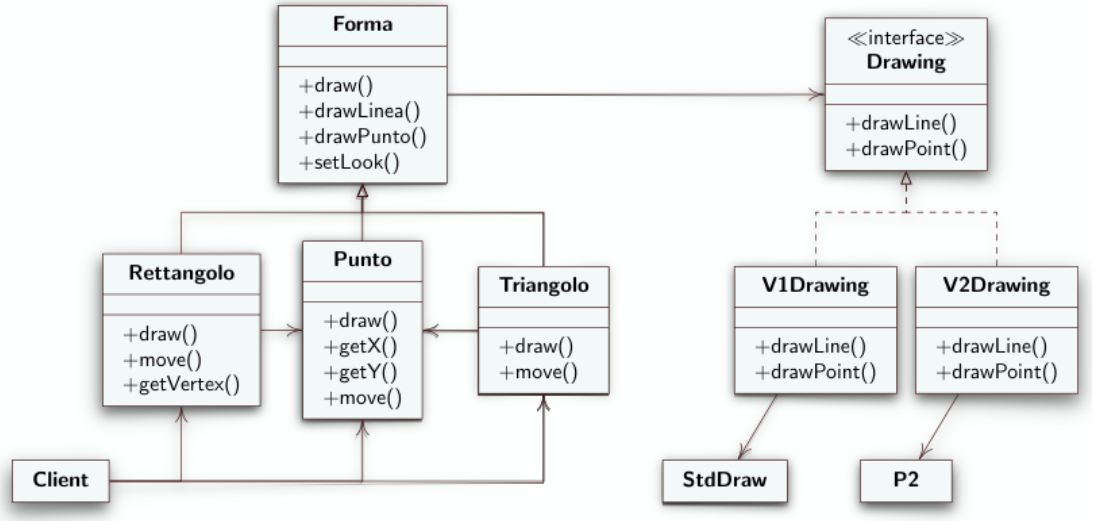


Figura 8.11: Vertici, primitive di linea e punto e due implementazioni grafiche indipendenti dalle forme.